

G1 Branch Strategy & The Case for the New Stack

2026-07-31 · For Adrián · Which code goes on the REAL robot for which test, and what the tutor-feedback stack buys you — every number below is measured, nothing is promised.

1 Which branch for which real test

Real-robot test	Code to check out	Why
R1 re-baseline (next session, 10 runs)	<code>tag golden-doorvis</code> <code>git checkout golden-doorvis</code>	Gate 1 requires exact comparability with the golden window (24-jul, 5 consecutive clean runs, B→A 52–60 s). Non-negotiable by protocol.
R2–R4 (water GOV/BASE ABBA, FULLCUP, wrong-prior, sealed lid)	Whatever the <i>Results Acquisition Plan</i> assigns per test (golden and <code>main</code> ; R3b is explicitly the main-vs-golden promotion)	The plan already fixes code per experiment so arms stay comparable.
Real trial of the NEW stack (meta state machine + human channel)	<code>tutor-feedback-metareasoner-sim</code> (tip 0480988+)	The published, most complete snapshot: Renxi's spec batches 1+2, RETREAT v2, RETREAT-AT-EXIT, marker v3 (<code>c / m</code> keys). Identical content to the held local branch — use the <code>-sim</code> ref until the goto.log coordination publishes the original. Runs AFTER Gate 1, as its own gated experiment with a fresh baseline to compare against.

Mental rule: golden = MEASURE · main = THE PLAN · `-sim` branch = THE NEW WORK. Nothing new touches the robot before a fresh re-baseline exists to compare it against.

Config line for the new-stack real trial (machine flags are default-ON on the branch):

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && git checkout tutor-feedback-metareasoner-sim && G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_STATE=meta2_state_branch_real.json G1_M2_STATE_INIT=meta2_state_twin_explored.json G1_M2_REALCAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_D0ORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.2 5 G1_M2_HELP_S=16 G1_D0OR_VIS=1 G1_PERC=192.168.51.80:8008 /Users/adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_goto.py goto B
```

Marker v3 in a second terminal (`python3 spill_mark.py`): ENTER=spill · `w <g>`=weigh · `i`=invalid · `c`=collision the robot missed · `m <cm>`=you moved/helped the robot.

2 What the new stack buys you (measured)

#	Advantage	The number behind it
1	Prevention under degraded sensing. Conservative caps in blind/noisy zones stop wedges from forming at all.	Closed loop under calibrated noise: OFF 3/6 reached, 27 collisions — ON 6/6, 0 collisions
2	Near-zero-cost insurance when things are fine. It does not touch good runs.	0 false interventions in clean closed loop; golden window replay 10/10 untouched; 2/192 (1.0%) false positives over all clean real runs. Price: +8.6% time (93→101 s median)
3	Structured recovery instead of grinding. Breadcrumb retreat along the known-good corridor.	Real data: 58% of stuck episodes NEVER self-freed; the ones that did took median 26 s (p75 52 s). The retreat undoes 0.7–

		0.8 m in ~14 s; 15/18 executed retreats un-wedged the robot (c0 0.3 → >0.75 m)
4	Human-in-the-loop rescue. After 3 failed retreats the robot STOPS and asks; your <code>m <cm></code> re-arms budgets and it resumes.	Demonstrated twice in one twin run. Retrospective over your real history: 35 runs would have asked for help; 48 minutes of post-request wrestling were cuttable. In real sessions a human is always present
5	Laser lie detection. The robot finally knows when its LiDAR deceived it — and you can tell it (c) about hits it missed.	55% of all real collisions (107/193) happened with the laser reading clear >0.6 m
6	Start-pocket protection. Retreats can now arm right at the waypoint exit (the "B nose-to-sofa" geometry).	Wedge draws: 1–2 collisions with structured close vs 7–14 grinding before; retreats armed at 0.40–0.47 m from start (previously impossible)
7	Evidence-grade observability. Every run logs <code>meta_state</code> , <code>iface_q</code> , <code>laser_trust</code> , <code>door_contra</code> + transition events.	Renxi's meta-spec implemented 1:1; every run analysable with <code>analyze_msm.py</code> / replayable with <code>replay_msm.py</code> — thesis-ready evidence for Ch.5

3 Honest caveats (say them before Renxi does)

- **Twin-validated, not real-validated.** All closed-loop numbers are from the digital twin under noise calibrated against your real 24-jul distributions. The real trial is exactly what the post-Gate-1 experiment is for.
- **+8–10% time in clean conditions** — the price of the BLIND caps. If R1 shows the real robot in golden form, that cost buys insurance you may rarely need; under bad sensing days it buys the difference between 6/6 and 3/6.
- **ASSIST does not latch** (it can drop out if clearance rises during an align dance). Worked correctly in every test, but the latched semantics is a known v2 item.
- **DEGRADED needs ≥3 missed-collision reports in ~30 s** — deliberate (a high bar against noise), so you will rarely see it; the trust dips and recoveries below the threshold are logged anyway.
- **DOOR-VIS in the twin was a tie** (4/6 vs 4/6, same door times): the twin's door is not the bottleneck; the golden-window benefit came from real map-reality mismatch. Keep it ON on the real robot (zero measured cost).
- **The B-pocket root cause is physical** (waypoint recorded too close to furniture) — the new stack manages it; re-recording the waypoint or censusing the obstacle would remove it.

4 Suggested deployment sequence

1. **Session 1 — R1** on `golden-doorvis`: 10 runs, Gate 1. (Checklist: `docs/G1_R1_Session_Checklist.pdf`.)
2. **Gate 1 PASS** → continue the plan (R2 water GOV/BASE...) *or* insert the branch trial next if you and Renxi prefer capability evidence first.
3. **Branch trial session** on `tutor-feedback-metareasoner-sim`: repeat the SAME 10-run pattern as R1 (5+5, 247 g, weigh every leg) so the comparison is 1:1 against your fresh baseline. Practise the two new keys: `c` when you see a hit the robot missed, `m <cm>` after you physically help it during ASSIST.
4. **Compare** (reached %, collisions, spills/leg, time; then `analyze_msm.py` per run) and decide the merge with data on the table.

One-line answer to "which branch?": R1 on `golden-doorvis`; the plan's tests on what the plan says; the new stack's real debut on `tutor-feedback-metareasoner-sim`, after Gate 1, mirrored 1:1 against the fresh baseline.